

SET-A

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

Write a Java program that takes the total electricity units consumed by a household as input from the user and calculates the total bill based on the following conditions:

- For the first 100 units: \$0.15 per unit
- For the next 150 units: \$0.20 per unit
- For units above 250: \$0.30 per unit

If the user enters a negative number of units, your program should print "Invalid Unit". Otherwise, print the total bill.

Sample Input 1: 80	Sample Output 1 Total Bill: \$12.0
Sample Input 2: 180	Sample Output 2: Total Bill: \$31.0
Sample Input 3: -100	Sample Output 3: Invalid Unit
Explanation: For sample input 2, First 100 units = $100 \times 0.15 = \$15.0$ Next 80 units = $80 \times 0.20 = \$16.0$ Total = \$31.0	

SET-B

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

Write a Java program that takes the total working hours of a day laborer as input from the user and calculates his salary based on the following conditions:

- For the first 2 hours: \$1.27 per hour
- For the next 3 hours: \$1.66 per hour
- For hours above 5: \$2.05 per hour

If the user enters a negative number as an hour, your program should print "Invalid Hour". Otherwise, print the total salary.

Sample Input 1: 3	Sample Output 1: Total Salary: \$4.2
Sample Input 2: 7	Sample Output 2: Total Salary: \$11.62
Sample Input 3: -2	Sample Output 3: Invalid Hour
Explanation: For sample input 1, The total working hours is 3 so the total salary is $(2*1.27)+(1*1.66) = \$4.2$	

Solution & Rubric

CSE110 Summer 2026 Lab Exam - 0X Tentative Solutions and Rubrics

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

SET - [A]

SOLUTION

```
import java.util.Scanner;
public class Quiz1SetA{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int unit = sc.nextInt();
        double bill = 0.0;
        if (unit < 0) {
            System.out.println("Invalid Unit");
        } else {
            if (unit <= 100) {
                bill = unit * 0.15;
            } else if (unit <= 250) {
                bill = (100 * 0.15) + ((unit - 100) * 0.20);
            } else {
                bill = (100 * 0.15) + (150 * 0.20) + ((unit - 250) * 0.30);
            }
            System.out.println("Total Bill: $" + bill);
        }
    }
}
```

RUBRIC

SL	Points to Meet	Marks [10]
1	Importing scanner class and Java syntax	1
2	Taking unit input	1
3	Declaring bill variable	1
4	Checking and printing for invalid hour	1.5
5	Checking conditions for 3 types of unit	0.5 + 0.5 + 0.5
6	Calculating salary for 3 types of unit	1+1+1
7	Proper print statement	1
Total		10

SET - [B]**SOLUTION**

```

import java.util.Scanner;
public class Quiz1SetB{
    public static void main (String [] args){
        Scanner sc = new Scanner(System.in);
        int hour = sc.nextInt();
        double salary = 0.0;
        if (hour<0){
            System.out.println("Invalid Hour.");
        }
        else{
            if (hour<=2){
                salary = 1.27*hour;
            }
            else if (hour<=5){
                salary = (2*1.27) + ((hour-2)*1.66);
            }
            else{
                salary = (2*1.27) + (3*1.66) + ((hour-5)*2.05);
            }
            System.out.println("Total Salary: $" + salary);
        }
    }
}

```

RUBRIC

SL	Points to Meet	Marks [10]
1	Importing scanner class and Java syntax	1
2	Taking hour input	1
3	Declaring salary variable	1
4	Checking and printing for invalid hour	1.5
5	Checking conditions for 3 types of hours	0.5 + 0.5 + 0.5
6	Calculating salary for 3 types of hours	1+1+1
7	Proper print statement	1
Total		10

SET-A

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

One day, Messi was solving a puzzle while relaxing after practice. He came across a challenge: There's a magical sequence called the Fibonacci sequence. It starts with 0 and 1, and every new number is the sum of the two before it. The sequence looks like this,

0, 1, 1, 2, 3, 5, 8, 13, 21, 34... and so on

Your task: Within the first 20 numbers of this sequence, find and print the first number that can be evenly divided by Messi's lucky number. If such number is not found, then print "Game Over". Take Messi's lucky number as input from the user.

Sample Case 1	Sample Case 2	Sample Case 3
Sample Input: Lucky_Number = 3	Sample Input: Lucky_Number = 7	Sample Input: Lucky_Number = 15
Expected Output: If Messi's lucky number is 3, the answer is 3	Expected Output: If Messi's lucky number is 7, the answer is 21	Expected Output: Game Over
Explanation: The first Fibonacci number that is evenly divisible by 3 is 3. Therefore, the output is: 3	Explanation: Among the first 20 Fibonacci numbers, 21 is the first number that is evenly divisible by 7. Therefore, the output is: 21	Explanation: None of these numbers is evenly divisible by 15. Therefore, the output is: Game Over

SET-B

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

One day, Cristiano Ronaldo was practicing his free kicks and found an interesting number challenge. There's a magical sequence called Fibonacci. It starts with 0 and 1, and every new number is the sum of the previous two numbers. The sequence looks like this:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ... and so on

Your Task: Within the first 15 numbers of the Fibonacci sequence, find and print the first number that is a multiple of Ronaldo's jersey number. If no such number exists within the first 15 numbers, print "No Goal". Take Ronaldo's jersey number as input from the user.

Sample Case 1	Sample Case 2	Sample Case 3
Sample Input: Jersey_Number = 5	Sample Input: Jersey_Number = 4	Sample Input: Jersey_Number = 22
Expected Output: If Ronaldo's jersey number is 5, the answer is 5	Expected Output: If Ronaldo's jersey number is 4, the answer is 8	Expected Output: No Goal
Explanation: The first Fibonacci number that is a multiple of 5 is 5. Therefore, the output is: 5	Explanation: Among the first 15 Fibonacci numbers, 8 is the first number that is a multiple of 4. Therefore, the output is: 8	Explanation: None of these numbers is a multiple of 22. Therefore, the output is: No Goal

Solution & Rubric

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

SET - [A]

SOLUTION

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Lucky_Number = ");
        int luckyNumber = sc.nextInt();

        int a = 0;
        int b = 1;
        boolean found = false;

        for (int i = 0; i < 20; i++) {
            if (a != 0 && a % luckyNumber == 0) {
                System.out.println("If Messi's lucky number is "+luckyNumber+", the answer is "+a);
                found = true;
                break;
            }
            int next = a + b;
            a = b;
            b = next;
        }

        if (!found) {
            System.out.println("Game Over");
        }
        sc.close();
    }
}
```

RUBRIC

SL	Points to Meet	Marks [10]
1	Java syntaxes (i.e. class, static void main, semicolon, curly braces and parentheses)	1
2	Declaring and initiating one variable mentioned in test data	1
3	Checking using loop	4
4	Checking the number using condition	3
5	Print statements	1

SET - [B]**SOLUTION**

```

import java.util.Scanner;
public class RonaldoChallenge {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Jersey_Number = ");
        int jersey = sc.nextInt();
        int first = 0;
        int second = 1;
        int count = 1;
        boolean found = false;

        while (count <= 15) {
            int current;
            if (count == 1) {
                current = 0;
            } else if (count == 2) {
                current = 1;
            } else {
                current = first + second;
                first = second;
                second = current;
            }
            if (current != 0 && current % jersey == 0) {
                System.out.println("If Ronaldo's jersey number is " + jersey + ", the answer is " + current);
                found = true;
                break;
            }
            count++;
        }
        if (!found) {
            System.out.println("No Goal");
        }
    }
}
/* Set B can be solved in a similar way as Set A. The same Fibonacci logic applies; only the input
name, output message, loop limit (first 15 Fibonacci numbers), and "No Goal" output need to be
changed. */

```

RUBRIC

SL	Points to Meet	Marks [10]
1	Java syntaxes (i.e. class, static void main, semicolon, curly braces and parentheses)	1
2	Declaring and initiating one variable mentioned in test data	1
3	Checking using loop	4
4	Checking the number using condition	3
5	Print statements	1

SET-A

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

Airport Luggage Tag

At an airport, a luggage tag may contain letters, digits, spaces, and special characters. The security system checks the tag and creates a Verified Tag along with a Risk Score.

Write a Java program that takes a luggage tag as input and processes it from left to right using the following rules:

For every English letter (A-Z or a-z), uppercase and lowercase versions of the same letter are considered identical. If the letter has not appeared before, add its uppercase version to the Verified Tag. Otherwise, do not add it again and increase the Risk Score by 3.

For every digit (0-9), add its numeric value to the Risk Score. Each digit is processed individually.

Ignore spaces and all special characters.

Security Status

Risk Score 0-7: Cleared

Risk Score 8-14: Manual Review

Risk Score 15 or more: High Risk

Print the Verified Tag, Risk Score, and Security Status.

Restrictions

The input will contain at least one English letter.

You may use only `.length()` and `.charAt()` as String methods.

Arrays and user-defined methods are not allowed.

Sample Input	Sample Output	Explanation
Sample Case 1 C4S-E1@11	Verified Tag: CSE Risk Score: 7 Security Status: Cleared	C, S, and E form the tag. The digits add $4 + 1 + 1 + 1 = 7$. The symbols are ignored, so this tests the highest Cleared score.
Sample Case 2 aA5#	Verified Tag: A Risk Score: 8 Security Status: Manual Review	The first a adds A. The next A is a case-insensitive duplicate (+3), and digit 5 adds 5. Total risk is 8, the lowest Manual Review score.
Sample Case 3 Ba4g-Ga23#	Verified Tag: BAG Risk Score: 15 Security Status: High Risk	B, A, and G are unique. Digit values add 9, while duplicate G and A add 6. Total risk is 15, the lowest High Risk score.

SET-B

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

Space Message Decoder

A spacecraft receives a coded message containing letters and a few control symbols. The message must be decoded before it can be read.

Write a Java program that takes the coded message as input and processes it from left to right using the following rules:

An English letter (A-Z or a-z) is added to the Decoded Message exactly as it appears.

An underscore (_) is added as a space.

The symbol ^ changes the case of the immediately following English letter and adds the changed letter to the Decoded Message. You may assume that ^ is always followed by an English letter. Add 2 to the Interference Score whenever ^ is processed.

A digit from 1 to 9 repeats the most recently added letter in the Decoded Message that many additional times. Add the digit's numeric value to the Interference Score. Every digit will appear immediately after an English letter in the coded input.

A period (.) ends the transmission. Stop processing immediately, even if more characters remain in the input.

Ignore all other special characters.

Signal Status

Interference Score 0-4: Stable

Interference Score 5-9: Unstable

Interference Score 10 or more: Critical

Print the Decoded Message, Interference Score, and Signal Status.

Restrictions

You may use only `.length()` and `.charAt()` as String methods.

Arrays and user-defined methods are not allowed.

Sample Input	Sample Output	Explanation
Sample Case 1 C^s2E_go@.XYZ9	Decoded Message: CSSSE go Interference Score: 4 Signal Status: Stable	^s adds S and 2 points; digit 2 adds two more S characters and 2 points. _ becomes a space, @ is ignored, and everything after the period is ignored.
Sample Case 2 m^I3ssion.	Decoded Message: miiiission Interference Score: 5 Signal Status: Unstable	^l adds lowercase i and 2 points. Digit 3 repeats that decoded i three additional times and adds 3 points. The final score is 5.
Sample Case 3 ^A2b^C4_x.#IGNORE	Decoded Message: aaabccccc x Interference Score: 10 Signal Status: Critical	^A2 produces aaa, while ^C4 produces ccccc. The underscore adds a space, and decoding stops at the period. The score is 2 + 2 + 2 + 4 = 10.

Solution & Rubric

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

SET - [A]

SOLUTION

```
import java.util.Scanner;

public class SetA {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String input = sc.nextLine();
        String verifiedTag = "";
        int riskScore = 0;

        for (int i = 0; i < input.length(); i++) {
            char ch = input.charAt(i);

            if ((ch >= 'A' && ch <= 'Z') ||
                (ch >= 'a' && ch <= 'z')) {

                if (ch >= 'a' && ch <= 'z') {
                    ch = (char)(ch - 32);
                }

                boolean duplicate = false;

                for (int j = 0; j < verifiedTag.length(); j++) {
                    if (verifiedTag.charAt(j) == ch) {
                        duplicate = true;
                        break;
                    }
                }

                if (duplicate) {
                    riskScore += 3;
                }
                else {
                    verifiedTag += ch;
                }
            }
            else if (ch >= '0' && ch <= '9') {
                int digit = ch - '0';
                riskScore += digit;
            }
        }

        System.out.println("Verified Tag: " + verifiedTag);
        System.out.println("Risk Score: " + riskScore);

        if (riskScore <= 7) {
            System.out.println("Security Status: Cleared");
        }
        else if (riskScore <= 14) {
            System.out.println("Security Status: Manual Review");
        }
        else {
            System.out.println("Security Status: High Risk");
        }
    }
}
```

RUBRIC

SL	Points to Meet	Marks [10]
1	Valid Java structure, syntax, and Scanner setup; reads the luggage tag correctly	1
2	Traverses the complete input from left to right using length() and charAt()	1
3	Handles letters case-insensitively, manually converts new letters to uppercase, and builds the Verified Tag without duplicates	3
4	Adds 3 for duplicate letters, adds each digit's numeric value individually, and ignores spaces and special characters	3
5	Applies the Cleared, Manual Review, and High Risk score boundaries correctly	1
6	Prints the Verified Tag, Risk Score, and Security Status in the required format	1

SET - [B]**SOLUTION**

```
import java.util.Scanner;
public class SetB {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String input = sc.nextLine();
        String decodedMessage = "";
        int interferenceScore = 0;
        char previousLetter = ' ';

        for (int i = 0; i < input.length(); i++) {
            char ch = input.charAt(i);

            if (ch == '.') {
                break;
            }
            else if (ch == '^') {
                char next = input.charAt(i + 1);

                if (next >= 'a' && next <= 'z') {
                    next = (char)(next - 32);
                }
                else if (next >= 'A' && next <= 'Z') {
                    next = (char)(next + 32);
                }

                decodedMessage += next;
                previousLetter = next;
                interferenceScore += 2;
                i++;
            }
            else if ((ch >= 'A' && ch <= 'Z') ||
                    (ch >= 'a' && ch <= 'z')) {
                decodedMessage += ch;
                previousLetter = ch;
            }
            else if (ch >= '1' && ch <= '9') {
                int repeat = ch - '0';

                for (int j = 0; j < repeat; j++) {
                    decodedMessage += previousLetter;
                }

                interferenceScore += repeat;
            }
            else if (ch == '_') {
                decodedMessage += " ";
            }
        }

        System.out.println("Decoded Message: " + decodedMessage);
        System.out.println("Interference Score: " + interferenceScore);

        if (interferenceScore <= 4) {
            System.out.println("Signal Status: Stable");
        }
        else if (interferenceScore <= 9) {
            System.out.println("Signal Status: Unstable");
        }
        else {
            System.out.println("Signal Status: Critical");
        }
    }
}
```

RUBRIC		
SL	Points to Meet	Marks [10]
1	Processes the message from left to right and stops immediately when a period is encountered	1
2	Processes the message from left to right and stops immediately when a period is encountered	2
3	Copies normal letters, converts underscores to spaces, and ignores other special characters	1
4	Changes the case of the letter after ^, stores the decoded letter, skips the consumed input letter, and adds 2 to the score	2
5	Repeats the most recently added decoded letter the required additional times and adds the digit value to the score	3
6	Prints the Decoded Message, Interference Score, and correct Signal Status	1